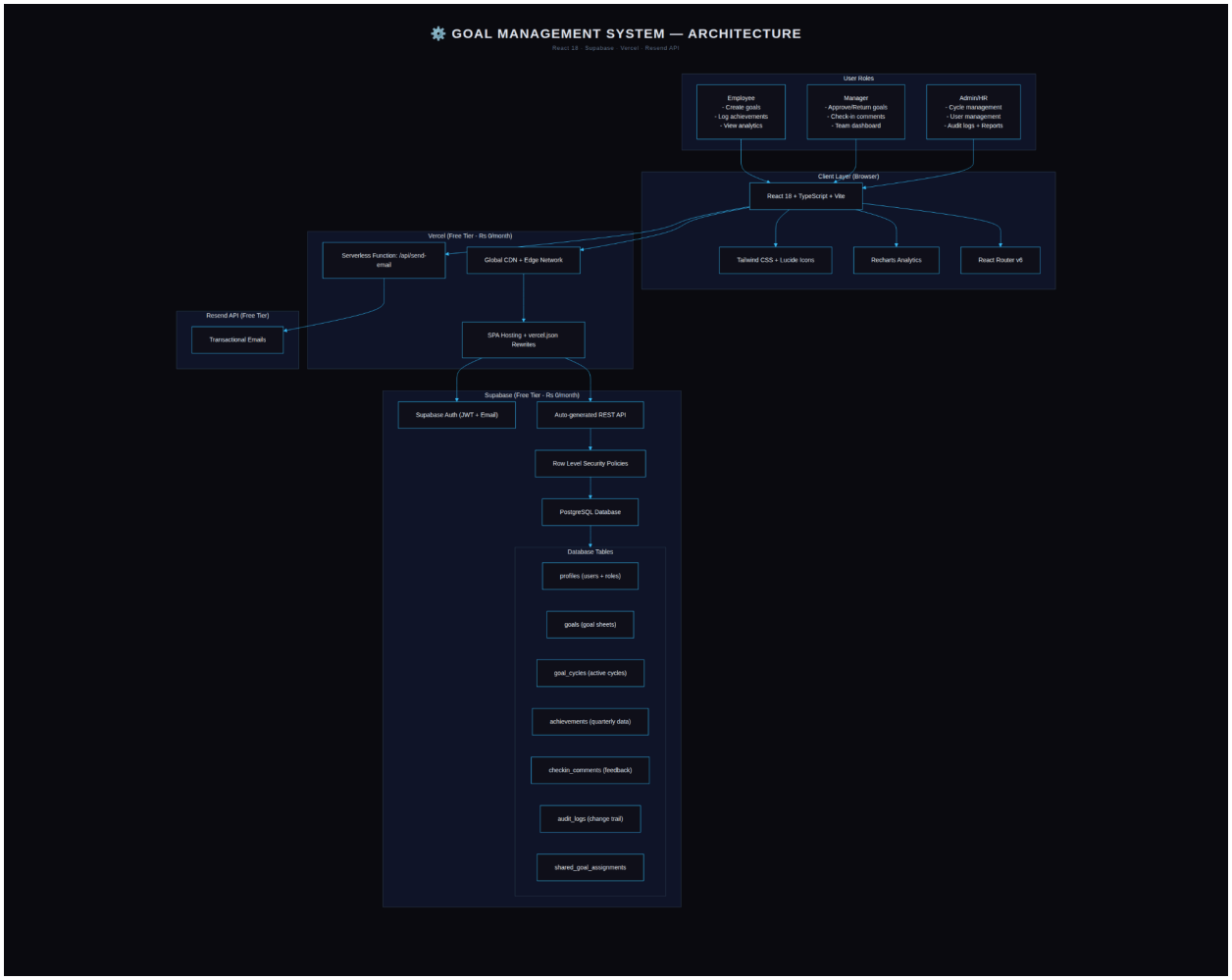
KaamKaaj — Goal Setting & Tracking Portal
System Architecture



KaamKaaj — Goal Setting & Tracking Portal

Hackathon Submission Document

Project: Corporate goal-setting and quarterly performance tracking portal
Author: Shivangi Singh
Stack: React 19 · TypeScript · Vite · Tailwind CSS · Supabase (PostgreSQL + Auth + RLS)

1. Working Application Link

Production (Vercel):
https://kaam-kaaj-blue.vercel.app

Notes: Frontend hosted on Vercel; backend on Supabase.

Deployment Instructions

Build and deploy using:

npm run build
npx vercel --prod

In the Vercel Dashboard → Project → Settings → Environment Variables, configure:

- VITE_SUPABASE_URL
- VITE_SUPABASE_ANON_KEY

Both values can be obtained from:

Supabase → Project Settings → API

After adding environment variables, redeploy the project.

In Supabase → Authentication → URL Configuration, configure:

Site URL:
https://kaam-kaaj-blue.vercel.app

Redirect URL:
https://kaam-kaaj-blue.vercel.app/**

Demo Login Credentials

Admin Login
Email: admin@demo.com
Password: Admin@123

Manager Login
Email: manager1@demo.com
Password: Manager@123

Employee Login
Email: employee1@demo.com
Password: Employee@123

Demo Data:
The employee account contains seeded goals for revenue (numeric_min) and response-time (numeric_max) with Q1 achievements showing 100% score calculation.

2. Source Code Repository

GitHub Repository:
https://github.com/shivangiS04/KaamKaaj-

Main Branch:
https://github.com/shivangiS04/KaamKaaj-/tree/main

Important Files:

- supabase/hackathon_critical_fixes.sql
- docs/hackathon-demo-scripts.md

Repository Structure

```
KaamKaaj-/
├── src/                # React frontend (pages, components, utilities)
├── supabase/           # Database schema, seeds, SQL migrations
├── docs/               # Architecture docs and demo scripts
├── package.json
└── vercel.json         # SPA routing configuration
```

3. Architecture Overview

The application uses a modern full-stack architecture built with React, TypeScript, Supabase, and PostgreSQL.

Frontend:

- React 19
- TypeScript
- Vite
- Tailwind CSS
- React Router
- Context API for authentication state management

Backend:

- Supabase Authentication
- PostgreSQL database
- Row-Level Security (RLS)
- PostgREST APIs
- Database triggers and stored procedures

Hosting:

- Frontend deployed on Vercel
- Backend hosted on Supabase

Core modules include:

- Employee Dashboard
- Manager Dashboard
- Admin Dashboard
- Goal Management
- Achievement Tracking
- Shared Goal Assignments
- Audit Logging

Achievement Flow

1. Employee enters quarterly achievement values.
2. Frontend calculates score using centralized score logic.
3. Data is sent to Supabase APIs.
4. Database validates whether the check-in window is open.
5. Achievement is stored in PostgreSQL.
6. Shared achievements are synchronized automatically.
7. Updated scores are returned to the frontend.

If the check-in window is closed, the request is rejected by the database trigger.

Role-Based Access Control

Employee Access:

- Own goals
- Personal achievements
- Shared goals assigned to them

Manager Access:

- Team goals
- Shared assignments
- Team performance visibility

Admin Access:

- Full system-wide access
- Configuration management
- Administrative controls

Security is enforced using PostgreSQL Row-Level Security policies.

4. Hackathon Critical Fixes

Issue 1: Incorrect numeric score calculations
Solution: Introduced centralized scoreCalculator.ts and database function calculate_achievement_score().

Issue 2: Shared goals existed only at schema level
Solution: Added Share Goal modal, Approved Goals page, read-only shared goals section, and synchronization triggers.

Issue 3: API allowed check-in bypass
Solution: Added enforce_checkin_window database trigger with admin bypass support.

Verification:

- 5 database functions implemented
- 3 database triggers added
- Shared goal synchronization working successfully
- Demo seed data validates correct scoring behavior

5. Technology Stack

Frontend:

- React 19
- TypeScript
- Tailwind CSS
- Lucide React

Build Tool:

- Vite 5

Routing and State Management:

- React Router v7
- React Context API

Backend:

- Supabase
- PostgreSQL 15+

Authentication and Security:

- Supabase Auth
- Row-Level Security (RLS)

Hosting:

- Vercel

Charts:

- Recharts (optional)

Conclusion

KaamKaaj is a corporate goal-setting and quarterly performance tracking portal designed to simplify employee performance management through secure role-based access, automated score calculation, and scalable backend architecture.

The project demonstrates:

- Full-stack application development
- Secure PostgreSQL database design
- Role-based access control using RLS
- Automated backend workflows with triggers
- Modern frontend engineering using React and TypeScript
- Production deployment using Vercel and Supabase

The platform is fully functional with working authentication, seeded demo data, and complete end-to-end performance tracking workflows.